

Homework 7

Isaac Adoboe - ieadoboe@mun.ca

Question 1

```
In [2]: # Import libraries
import numpy as np
from datetime import datetime
```

```
In [3]: # Define function
def f(x): return np.sin(x**2)

# Function returns python lists
def py_list_eval(nsamples, function=f):
    x_list = []
    f_list = []

    x = np.linspace(0, 1, nsamples+1)
    f_x = function(x)

    for x_i in x:
        x_list.append(x_i)
    for f_i in f_x:
        f_list.append(f_i)

    return x_list, f_list

# Function returns python ndarrays
def py_ndarray_eval(nsamples, function=f):
    x = np.linspace(0, 1, nsamples+1)
    f_x = function(x)
    return x, f_x
```

```
In [4]: py_list_eval(nsamples=2)
```

```
Out[4]: ([np.float64(0.0), np.float64(0.5), np.float64(1.0)],
         [np.float64(0.0),
          np.float64(0.24740395925452294),
          np.float64(0.8414709848078965)])
```

```
In [5]: py_ndarray_eval(nsamples=2)
```

```
Out[5]: (array([0. , 0.5, 1. ]), array([0. , 0.24740396, 0.84147098]))
```

```
In [6]: print("For python lists")
print("-"*50)
for x in range(9):
    nsample = 10**x
    start = datetime.now()
    py_list_eval(nsample=nsample)
    end = datetime.now()
    elapsed = (end-start).total_seconds()
    print(f"For {nsample} number of samples, it took {elapsed:.8f} sec
```

For python lists

```
-----
For 1 number of samples, it took 0.00020100 seconds
For 10 number of samples, it took 0.00004300 seconds
For 100 number of samples, it took 0.00351600 seconds
For 1000 number of samples, it took 0.00014200 seconds
For 10000 number of samples, it took 0.00078000 seconds
For 100000 number of samples, it took 0.01395900 seconds
For 1000000 number of samples, it took 0.08624000 seconds
For 10000000 number of samples, it took 0.81605500 seconds
For 100000000 number of samples, it took 9.93897900 seconds
```

```
In [7]: print("For python np.ndarray")
print("-"*50)
for x in range(9):
    nsample = 10**x
    start = datetime.now()
    py_ndarray_eval(nsample=nsample)
    end = datetime.now()
    elapsed = (end-start).total_seconds()
    print(f"For {nsample} number of samples, time: {elapsed:.8f} sec
```

For python np.ndarray

```
-----
For 1 number of samples, time: 0.00005300 seconds
For 10 number of samples, time: 0.00002000 seconds
For 100 number of samples, time: 0.00001100 seconds
For 1000 number of samples, time: 0.00001600 seconds
For 10000 number of samples, time: 0.00055300 seconds
For 100000 number of samples, time: 0.00117300 seconds
For 1000000 number of samples, time: 0.00541100 seconds
For 10000000 number of samples, time: 0.05427900 seconds
For 100000000 number of samples, time: 0.54096300 seconds
```

Comments:

- For 1 million samples, numpy (ndarray version) is over 8 times faster (0.0103s vs 0.0825s) as compared to the python lists version. This ratio widens as `nsample` increases
- At smaller values of `nsample` (`nsample=1`, `nsample=10` & `nsample=100`), the lists looping approach can be faster than (or

comparable to) the numpy ndarray approach.

- It is clear that for very large values of `nsample`, the benefits of the numpy ndarrays become clear.
- For trivial problem sizes, the implementation choice doesn't really matter. For realistic scientific computing problems (sizes), numpy arrays has the most performance benefits and are highly recommended.

Question 2

```
In [8]: # Import libraries
import numpy as np
import scipy.special as special
import matplotlib.pyplot as plt
from common import mpl_apply
```

```
In [9]: # Apply mpl config
mpl_apply()
```

```
In [10]: markers = ["x", "v", "s", "o", "*"]

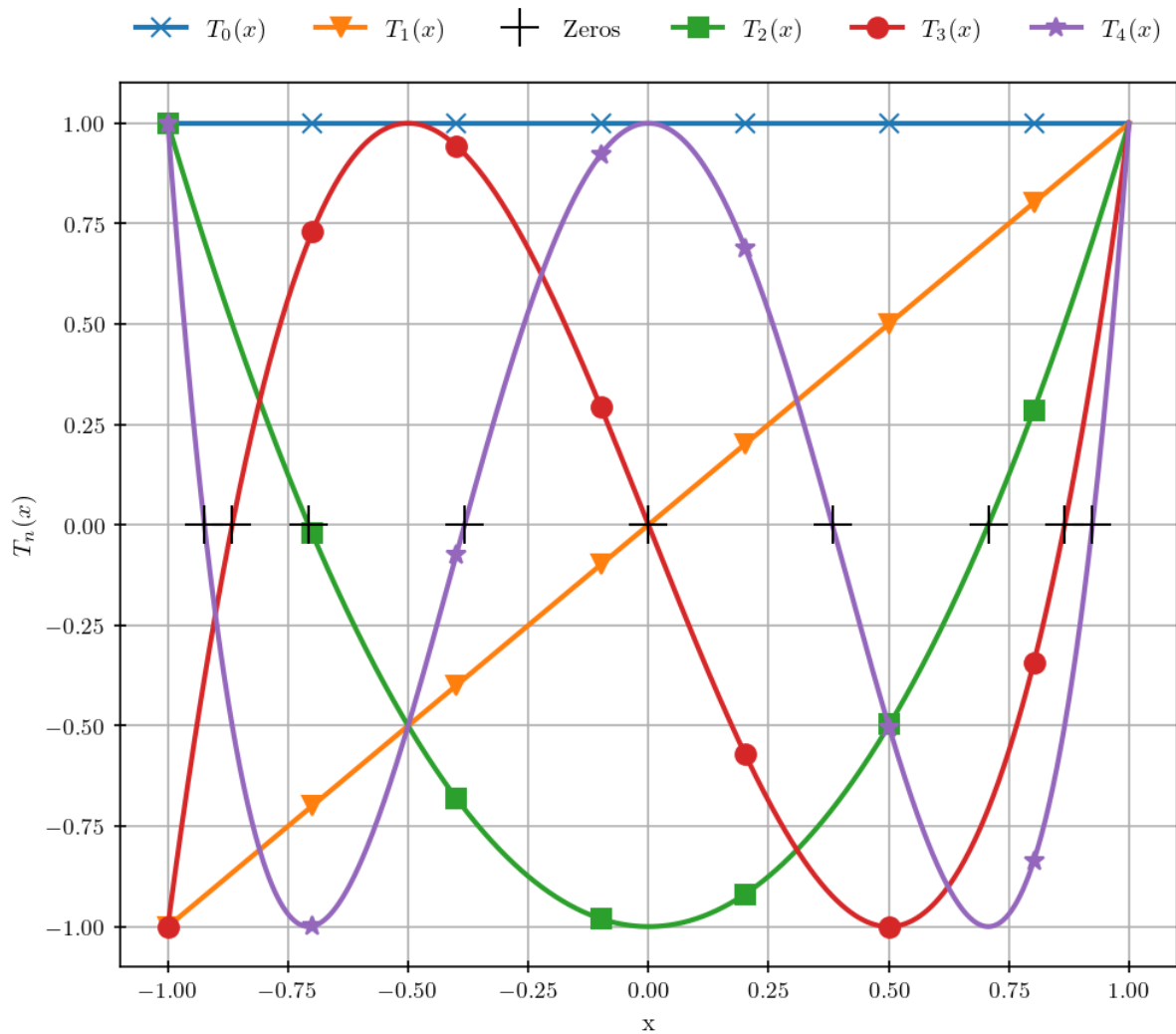
x = np.linspace(-1, 1, 1000)
zero_plotted = False

plt.figure(figsize=(7, 6))
for n in range(5):
    y = special.eval_chebyt(n, x)
    plt.plot(x, y, label=f"$T_{n}(x)$",
             marker=markers[n], markevery=150, markersize=8)

    if n > 0: # T_0 has no zeros
        zeros = np.cos((2*np.arange(1, n+1) - 1) * np.pi / (2*n))
        # Only add label once to avoid duplicate legend entries
        label = "Zeros" if not zero_plotted else None
        plt.plot(zeros, np.zeros_like(zeros),
                 'k+', markersize=15, label=label)
        zero_plotted = True

plt.xlabel('x')
plt.ylabel('$T_n(x)$')

plt.legend(loc='upper center', bbox_to_anchor=(
    0.5, 1.1), ncol=6, frameon=False)
plt.tight_layout()
plt.grid()
```



Question 3

```
In [11]: # Import libraries
import numpy as np
import matplotlib.pyplot as plt
import scipy.interpolate as interp
```

interp1d

```
In [12]: # Function
def f(x):
    return np.e**((np.sin(np.pi*x)))

# Data points
x = np.linspace(0, 3, 15)
y = f(x)

x_fine = np.linspace(0, 3, 200)
```

```

# interpolants
g_nearest = interp.interp1d(x, y, kind='nearest')
g_linear = interp.interp1d(x, y, kind='linear')
g_cubic = interp.interp1d(x, y, kind='cubic')

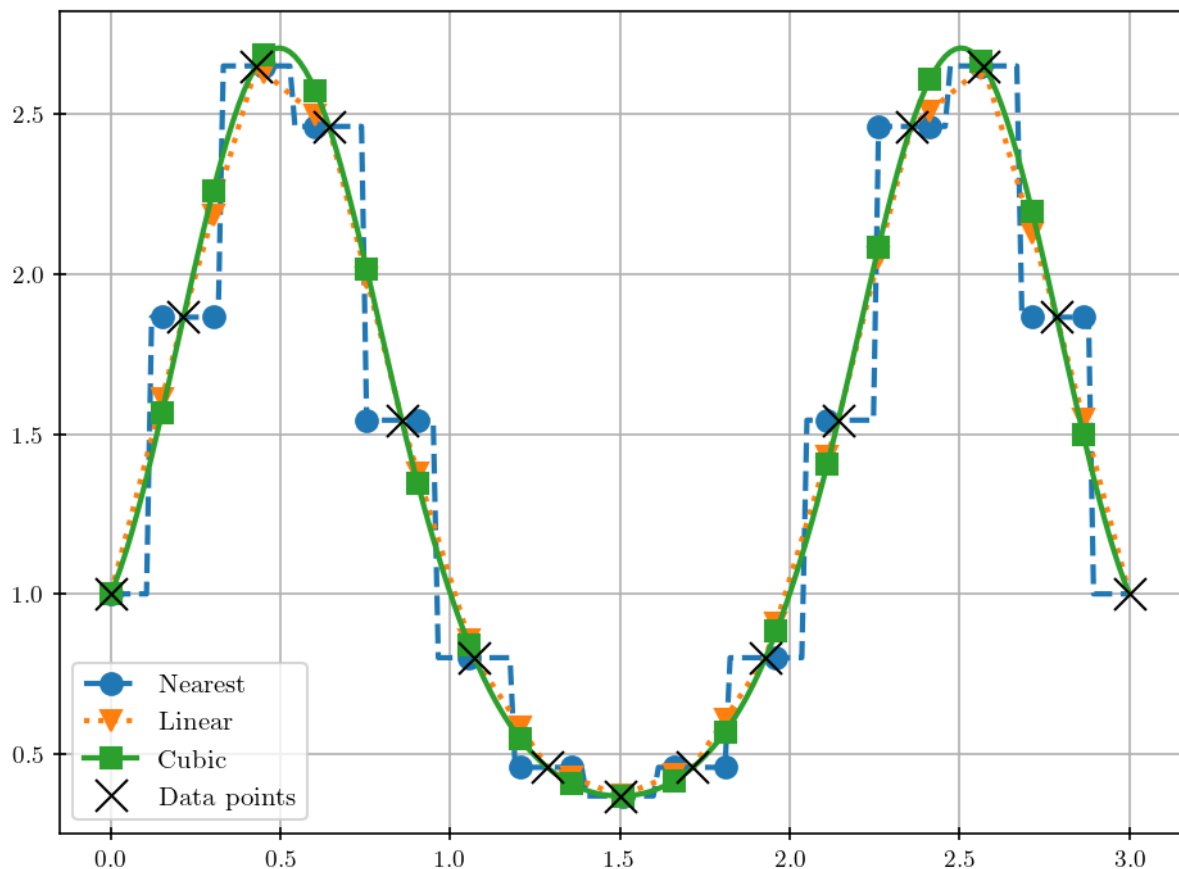
y_nearest = g_nearest(x_fine)
y_linear = g_linear(x_fine)
y_cubic = g_cubic(x_fine)

kwargs = {"markevery": 10,
          "markersize": 8}

# Plot
plt.plot(x_fine, y_nearest, label="Nearest",
         linestyle="--", marker="o", **kwargs)
plt.plot(x_fine, y_linear, label="Linear",
         linestyle=":", marker="v", **kwargs)
plt.plot(x_fine, y_cubic, label="Cubic",
         linestyle="--", marker="s", **kwargs)
plt.plot(x, y, 'kx ', label="Data points", markersize=12)

plt.legend()
plt.grid()
plt.tight_layout()

```



CubicSpline

```
In [13]: def f(x):
          return np.e**(np.sin(np.pi*x))

# Data points
x = np.linspace(0, 3, 15)
y = f(x)

# Sample points
x_fine = np.linspace(0, 3, 200)

# Exact derivative
def f_prime(x):
    return np.e**(np.sin(np.pi*x)) * np.cos(np.pi*x) * np.pi

d = f_prime(x_fine)

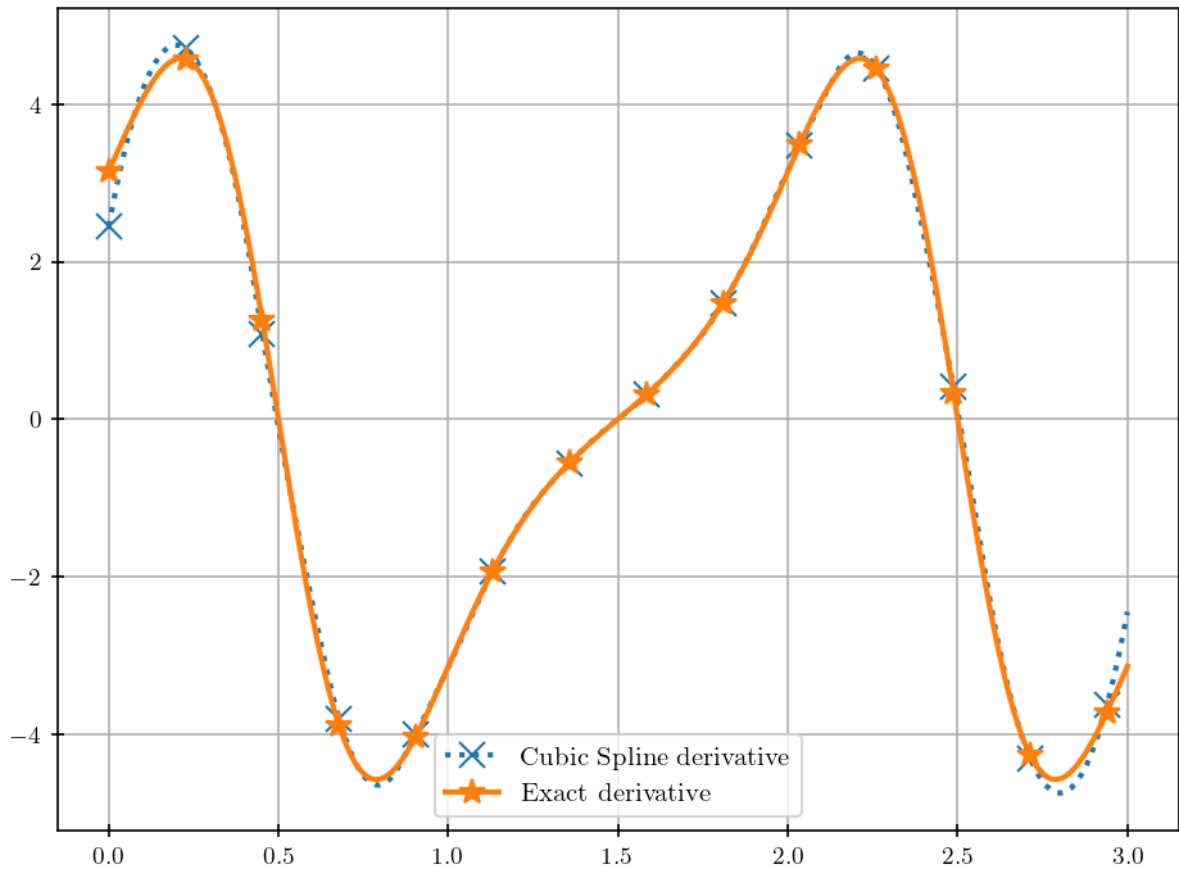
# Cubic spline
gspline = interp.CubicSpline(x, y, bc_type='not-a-knot')

# derivative function
dg = gspline.derivative()
d_fine = dg(x_fine)

kwargs = {"markevery": 15,
          "markersize": 10}

# Plot
plt.plot(x_fine, d_fine, label="Cubic Spline derivative",
         linestyle=":", marker="x", **kwargs)
plt.plot(x_fine, d, label="Exact derivative",
         linestyle="-", marker="*", **kwargs)

plt.legend()
plt.grid()
plt.tight_layout()
```



fixed_quad

```
In [14]: import scipy.integrate as integrate
import numpy as np
```

```
In [15]: def f(x):
return np.e**(np.sin(np.pi*x))
```

```
In [16]: for n in range(1, 21):
val = integrate.fixed_quad(f, 0, 3, n=n)
print(f"With order {n}, approximation is {val[0]}")
```

```

With order 1, approximation is 1.103638323514327
With order 2, approximation is 7.473298640663884
With order 3, approximation is 4.482331852412473
With order 4, approximation is 3.9367345442412
With order 5, approximation is 4.674223744064444
With order 6, approximation is 4.555405996923763
With order 7, approximation is 4.469432470895808
With order 8, approximation is 4.513436874383119
With order 9, approximation is 4.512000409516655
With order 10, approximation is 4.506793843844232
With order 11, approximation is 4.508517558242957
With order 12, approximation is 4.508595517848832
With order 13, approximation is 4.508388292755122
With order 14, approximation is 4.508440814062365
With order 15, approximation is 4.508445730325365
With order 16, approximation is 4.508439424373909
With order 17, approximation is 4.508440788439414
With order 18, approximation is 4.5084409443458995
With order 19, approximation is 4.508440786690819
With order 20, approximation is 4.508440818431111

```

```
In [17]: integrate.fixed_quad(f, 0, 3, n=20)
```

```
Out[17]: (np.float64(4.508440818431111), None)
```

```
In [18]: integrate.quad(f, 0, 3)
```

```
Out[18]: (4.508440819193944, 1.2587343389824923e-09)
```

Timings:

```
In [19]: %timeit integrate.fixed_quad(f, 0, 3, n=20)
```

```
6.5 µs ± 15.7 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops ea
ch)
```

```
In [20]: %timeit integrate.quad(f, 0, 3)
```

```
35.8 µs ± 135 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops eac
h)
```

Comments

- The difference between the two methods (`fixed_quad` vs `quad`) is very small ($< 10^{-8}$) after order 20 with `fixed_quad` . It is important to note that convergence was not immediate and took a few function evaluations to stabilize as seen for `fixed_quad` .
- `fixed_quad(n=20)` is more than 5 times faster than `quad` .
- That said, even though `fixed_quad` is faster, it requires that you know (have an idea) in advance the number of point evaluations you would need to

reach convergence.

Question 4

```
In [21]: # Import libraries
import numpy as np
import scipy.optimize as optim
```

```
In [22]: # Function
def f(vector):
    x, y = vector
    return (1000*x*y - 1)**2 + (np.e**(-x) + np.e**(-y) - 1.0001)**2

# Initial guess
x0 = np.array([0, 1])
print(x0)
```

```
[0 1]
```

```
In [33]: x0[1]
```

```
Out[33]: np.int64(1)
```

4(a) - BFGS without providing the Jacobian

```
In [32]: out = optim.minimize(f, x0,
                             tol=1e-9)
print(out)
```

```
message: Desired error not necessarily achieved due to precision loss.
success: False
status: 2
  fun: 1.0042211728039783e-06
   x: [ 1.477e-04  6.769e+00]
  nit: 47
  jac: [-5.261e+00 -1.317e-04]
hess_inv: [[ 1.526e-05 -6.866e-01]
            [-6.866e-01  3.092e+04]]
 nfev: 279
 njev: 89
```

4(b) - BFGS with providing the Jacobian

```
In [25]: def df(xy):
    x, y = xy
    u = 1000*x*y - 1
    v = np.e**(-x) + np.e**(-y) - 1.0001
```

```

df_dx = 2*u*1000*y + 2*v*(-np.e**(-x))
df_dy = 2*u*1000*x + 2*v*(-np.e**(-y))

return np.array([df_dx, df_dy])

```

```

In [26]: out = optim.minimize(f, x0, jac=df,
                             tol=1e-6)
print(out)

message: Optimization terminated successfully.
success: True
status: 0
      fun: 2.681683777365751e-23
       x: [ 1.187e-04  8.428e+00]
      nit: 120
      jac: [-4.065e-08 -5.703e-13]
hess_inv: [[ 2.450e-03 -1.740e+02]
            [-1.740e+02  1.236e+07]]
      nfev: 152
      njev: 152

```

4(c) - Newton-CG with providing the Jacobian but not the Hessian

```

In [27]: out = optim.minimize(f, x0, jac=df, method="Newton-CG",
                             tol=1e-10)
print(out)

message: Warning: CG iterations didn't converge. The Hessian is not positive definite.
success: False
status: 3
      fun: 3.3183427616618902e-06
       x: [ 1.620e-04  6.174e+00]
      nit: 93
      jac: [-7.514e+00 -2.045e-04]
      nfev: 224
      njev: 782
      nhev: 0

```

4(d) - Newton-CG with providing both the Jacobian and the Hessian

```

In [28]: def Hf(xy):
          x, y = xy
          u = 1000*x*y - 1
          v = np.e**(-x) + np.e**(-y) - 1.0001

          h_xx = 2*1000**2*y**2 + 2*np.e**(-2*x) + 2*np.e**(-x)*v
          h_yy = 2*1000**2*x**2 + 2*np.e**(-2*y) + 2*np.e**(-y)*v

```

```

h_xy = 2*1000**2*x*y + 2000*u + 2*np.e**(-(x+y))

return np.array([[h_xx, h_xy],
                  [h_xy, h_yy]])

```

```

In [29]: out = optim.minimize(f, x0, jac=df, hess=Hf, method="Newton-CG",
                             tol=1e-9)
print(out)

```

```

message: Optimization terminated successfully.
success: True
status: 0
  fun: 5.759385060328597e-07
   x: [ 1.448e-04  6.904e+00]
  nit: 116
  jac: [ 3.228e-11 -1.492e-06]
 nfev: 259
 njev: 259
nhev: 116

```

4(e) Nelder-Mead

```

In [30]: out = optim.minimize(f, x0, method='Nelder-Mead',
                              options={'maxfev': 500})
print(out)

```

```

message: Optimization terminated successfully.
success: True
status: 0
  fun: 6.830993530576365e-17
   x: [ 1.187e-04  8.428e+00]
  nit: 260
 nfev: 477
final_simplex: (array([[ 1.187e-04,  8.428e+00],
                        [ 1.187e-04,  8.428e+00],
                        [ 1.187e-04,  8.428e+00]]), array([ 6.831e-17,
1.084e-16,  3.003e-16]))

```

BFGS with Jacobian produced the best result, finding a minimum value of $[2.681683777365751e-23]$ at $(x^*, y^*) = (1.187e-04, 8.428e+00)$, which is essentially zero.

Nelder-Mead achieved comparable accuracy $6.830993530576365e-17$ but required significantly more function evaluations (477), while Newton-CG with derivatives reached only $5.759385060328597e-07$.

The critical finding is that providing derivatives is essential for this problem.